

Architectural Justification

Self-Service Coffee Kiosk System — Lab 3

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Two Reasons

1. Clear separation of concerns for a single, self-contained device. The kiosk's functionality splits naturally into a touch-screen presentation layer, order/payment/receipt business logic, and a data layer for menu and pricing storage. Layered architecture directly mirrors this structure, keeping the UI code independent of business rules and independent of how data is stored, which makes the system easier to build, test, and maintain within the 60-minute development scope of a single kiosk deployment.
2. Appropriate scale for the problem. The kiosk is a single, standalone device with a fixed, well-defined set of responsibilities — it does not need the independent scaling or fault isolation that Microservices architecture provides, and that complexity would introduce unnecessary operational overhead (multiple deployable services, network calls, inter-service data consistency) for what is fundamentally one machine running one application. A pure Client-Server split was also considered, but since all processing genuinely happens on the kiosk itself with no remote server dependency for order-taking, a layered structure better reflects the actual architecture without introducing a false single point of failure across a network.

Security Advantage

All payment-related logic is isolated inside a single Payment Service Component within the Business Layer. The User Interface Component never directly touches raw credit card data — it only interacts with the Order Manager Component through the Order Interface, and the Order Manager forwards payment requests through a well-defined Payment Interface. This layering confines sensitive payment data handling to one component, reducing the attack surface and making it straightforward to apply stricter security controls (e.g., encryption, PCI-compliant handling) to that single component rather than across the entire codebase.

Performance Benefit

Because all layers execute within the same kiosk device rather than communicating over a network (as Microservices or a networked Client-Server model would require), calls between the Order Manager, Payment Service, Receipt Printer, and Database components are direct, in-process calls rather than network round-trips. This eliminates network latency as a bottleneck and directly supports the requirement that a customer be able to complete an order — selection, payment, and receipt — in under 60 seconds.